

AI Intelligence Pipeline — Architecture Specification

Production-Grade, Fault-Tolerant, Deterministic Multi-Source Ingestion Engine

1. Executive Summary & System Scope

The **AI Intelligence Pipeline** is a high-throughput, resilient data platform engineered to ingest, clean, resolve, and publish AI market intelligence across five core verticals: **Startups, AI Products, Research Papers, Jobs, and News**. The system guarantees complete data provenance, zero hallucinations, strict 24-hour freshness verification, and automated canonicalization.

2. End-to-End Architectural Pipeline Flow

```
[ Sources: YC, HF, Arxiv, Techstars, VentureBeat, WorkAtAStartup ] ↓ [ Async Crawlers Layer ] -> aiohttp (Fast HTTP) + Playwright (Headless JS/SPA) ↓ [ Anti-Bot & Resiliency ] -> Bounded Concurrency + Jittered Exponential Backoff ↓ [ Distributed Deduplication ] -> Redis URL Canonicalizer + SHA-256 Fingerprinting ↓ [ Document Sanitation ] -> HTML Cleaner + Boilerplate Stripper (413 Prevention) ↓ [ Deterministic Extraction ] -> Multi-Tier Date Parser + GitHub Verified Stars ↓ [ Freshness Verification ] -> Strict 24-Hour Cutoff Enforcement (Jobs & News) ↓ [ Entity Resolution ] -> Deterministic Rule-Based Canonicalizer + Mapping Log ↓ [ Storage & Persistence ] -> PostgreSQL 16 ACID Storage (Upsert on Source URL) ↓ [ Verified Export ] -> Google Sheets 6-Tab Real-Time Sync + Audit Logs
```

3. Non-Negotiable Engineering Pillars

- **Zero Hallucinations:** Every single factual data point originates strictly from verified source URLs. Programmatic heuristics and API contracts are prioritized over LLM guesswork.
- **Immutable Provenance:** Records store original source URL, source name, extraction timestamps, and canonical resolution logs.
- **Deterministic Primacy:** Date extraction (JSON-LD → OpenGraph → time tags), URL normalization, GitHub star queries, and schema validations run on deterministic code.
- **413 & 429 Resilience:** Aggressive HTML boilerplate removal guarantees LLM context boundaries; per-source rate limiters and Retry-After headers avoid retry storms.

Core Subsystems & Technical Design

Component-Level Specifications & Deterministic Processing Protocols

4. Ingestion & Anti-Bot Crawling Strategy

The crawler subsystem employs a dual-engine architecture designed for bounded concurrency and polite scraping:

- **aiohttp Async HTTP Crawler:** High-performance async requests with connection pooling, DNS caching, and bounded concurrency (max 50 simultaneous sockets).
- **Playwright Headless Browser:** Isolated browser contexts executing Chromium for client-side rendered Single-Page Applications (SPA), with automated DOM ready-state waits.
- **Polite Rate Limiting & Jitter:** Decorator-based exponential backoff with full randomized jitter: $\text{delay} = \min(\text{max_delay}, \text{base} * (2 ** \text{attempt}) + \text{random_uniform})$. Respects HTTP 429 Retry-After headers.
- **Block Quarantine:** Immediate bail-out on permanent HTTP 403/401 blocks without blind retries, preserving error diagnostics in crawl audit logs.

5. Deterministic Extraction, Sanitization & Freshness

Subsystem	Mechanism	Guarantee
HTML Cleaner	Trafilatura + BeautifulSoup LXML AST parser; strips scripts, styles, navs, footers, SVGs.	Zero 413 Payload Errors; extracts semantic text & accurate token approximations.
Date Parser	Hierarchical multi-tier resolver: JSON-LD datePublished → OpenGraph → <time> tag → visible regex.	Produces timezone-aware UTC ISO-8601 strings; rejects ambiguous guesses.
Freshness Gate	Strict mathematical verification: $(\text{now_utc} - \text{record_date}) \leq 24.0 \text{ hours}$.	Applied to AI Jobs and AI News to eliminate stale postings.
GitHub Stars	Deterministic GitHub REST v3 API client with personal token authentication.	100% verified stargazer counts for research papers; zero LLM speculation.

6. Entity Resolution & Canonicalization

Eliminates messy entity variations (e.g. "OpenAI Inc.", "OpenAI, LLC", "Open AI" → "OpenAI"). Executes regex-driven punctuation stripping, corporate suffix removal, whitespace trimming, and alias mapping against a curated seed database of prominent AI organizations. All resolutions are recorded in the **Entity Mapping Log** with confidence scores, matching methods, and source traces.

Storage, Delivery & Operational Verification

ACID Persistence, Cloud Export, and Quality Assurance

7. Storage Engine & Data Contracts

PostgreSQL 16 serves as the primary system of record, utilizing connection pooling via `asyncpg`. Every entity table is protected by a `UNIQUE (source_url)` constraint, enforcing idempotent upserts:

- `startups`: Company name, description, website, founded date, batch, provenance.
- `products`: Product name, description, category, pricing structure, provenance.
- `research_papers`: Paper title, authors, publication date, arXiv link, GitHub URL, stars.
- `jobs`: Role title, company name, location, remote status, posting timestamp, apply URL.
- `news`: Headline, summary content, publication date, origin article URL.
- `entity_mappings`: Raw input name, resolved canonical name, matching rule, confidence score.

8. Google Sheets Cloud Exporter

Validated Pydantic entities are synchronized to Google Sheets via service account OAuth2 credentials. The exporter maintains strict idempotency by inspecting existing primary keys (`source_url` and `raw_name`) before issuing batched `append_rows` calls, with automatic worksheet creation and header initialization.

9. Test Coverage & Verification Matrix

Verification Area	Test Suite	Result & Metric
Unit & Integration Tests	<code>tests/</code> (pytest & pytest-asyncio)	100% Passing (227+ tests) across storage, crawlers, parser, and schemas.
Code Quality & Linting	<code>ruff check .</code>	Zero violations. Enforces PEP8, imports, naming, and bug prevention.
Static Type Analysis	<code>mypy src</code>	Zero errors. Type safety verified across all source files.
Bytecode Compilation	<code>compileall src</code>	Clean compilation without syntax or execution warnings.

10. CLI Deployment Commands

```
pipeline health
```

 — Validates live connectivity to PostgreSQL 16 and Redis 7.

```
pipeline run --category <name> --limit 100
```

 — Executes bounded crawl, ingestion, and Google Sheets export.

```
pipeline run --dry-run
```

 — End-to-end dry-run validation without mutation.